

LAB 7 (SECJ1023)
PROGRAMMING TECHNIQUE II
SECTION 03, 04 & 06, SEM 2, 2025/2026

1. Based on the Program 7.1, answer the following questions.

```
1 //Program 7.1
2 class Shape{
3     private:
4         double area;
5     public:
6         void setArea(double);
7         double getArea();
8 };
9
10 void Shape::setArea(double area){
11     this->area=area;
12 }
13
14 double Shape::getArea(){
15     return area;
16 }
17
18 class Circle:public Shape{
19     private:
20         double radius;
21     public:
22         void setRadius(double);
23         double getRadius();
24 };
25
26 void Circle::setRadius(double radius){
27     this->radius=radius;
28 }
29
30 double Circle::getRadius(){
31     return radius;
32 }
```

- a. Draw a UML class diagram to show the relationship between these two classes.
- b. When a `Circle` object is created, what are its public members?
- c. What members of the `Shape` class are not accessible to the `Circle` class's methods?
- d. Assuming a main method has the following declarations:

```
Shape s;
Circle c;
```

Indicate whether the following statements are legal or illegal:

```
c.setRadius(10.0);
s.setRadius(10.0);
cout<<c.getArea();
```

```
cout<<s.getArea();
```

2. A superclass Rectangle, subclass Square and main method are shown in Program 7.2. You are required to fill in the blanks with the correct instruction according to the numbered questions.

```

1 //Program 7.2
2 #include <iostream>
3 using namespace std;
4
5 class Rectangle {
6     protected:
7         double width;
8         double length;
9         double calculateArea();
10    public:
11        Rectangle(double, double);
12        void display();
13 };
14
15 Rectangle::Rectangle(double width, double length){
16     _____(a)_____;
17     _____(a)_____;
18 }
19
20 void Rectangle::display(){
21     cout<<width<<endl;
22     cout<<length<<endl;
23 }
24
25 double Rectangle::calculateArea(){
26     return width*length;
27 }
28
29 class Square_____(b)_____{
30     private:
31         double height;
32     public:
33         _____(c)_____;
34         double calculateVolume();
35         void display();
36 };
37
38 _____(d)_____{
39     this->height=height;
40     this->width=width;
41     this->length=length;
42 }
43
44 double Square::calculateVolume(){
45     return _____(e)_____;
46 }
47

```

```

48 void Square::display() {
49     _____(f)_____;
50     cout<<height;
51     cout<<calculateVolume();
52 }
53
54 int main() {
55     _____(g)_____;
56     _____(h)_____;
57     _____(i)_____;
58     return 0;
59 }

```

- a. Assign values for superclass attributes.
 - b. Subclass Square inherits from superclass Rectangle.
 - c. A function prototype for Square constructor that inherits Rectangle constructor.
 - d. A respective function header for Square constructor based on (c).
 - e. Calculate the volume for Square by using the inherited method.
 - f. Display the width and length of Square by using the inherited method.
 - g. Create a Square object named squareBox and pass the values of 15, 10, 10 for height, width and length respectively.
 - h. Calculate the volume of squareBox.
 - i. Display the properties of squareBox.
3. Examine the class in Program 7.3. Table 7.1 lists the variables that are members of the Words class (some are inherited). Complete the table by filling in the access specification each member will have in the Words class. Write “inaccessible” if a member is inaccessible to the Words class.

```

1  //Program 7.3
2  class Book {
3      private:
4          string name;
5      protected:
6          int page;
7      public:
8          string publisher;
9  };
10
11 class Dictionary:protected Book
12 {
13     private:
14         int numWords;
15     protected:
16         string meaning;
17     public:
18         string words;

```

```

19 };
20
21 class Words:public Dictionary
22 {
23     private:
24         int num;
25     protected:
26         string perkataan;
27     public:
28         string source;
29 };

```

Table 7.1: Access Specification for Member Variables in `Words` Class

Member Variable	Access Specification in <code>Words</code> Class
name	
page	
publisher	
numWords	
meaning	
words	
perkataan	
num	
source	

4. Given three classes named `Automobile`, `Car` and `Truck` respectively, and some objects of these classes as shown in Program 7.4, answer questions (a) to (c).

```

1  //Program 7.4
2  #include <iomanip>
3  #include <iostream>
4  using namespace std;
5
6  class Automobile
7  {
8      private:
9          string make;
10
11      protected:
12          int model;
13
14      public:
15          double price;
16
17          Automobile()
18          {
19              make = "";
20              model = 0;
21              price = 0.0;
22              cout << "An Automobile object has been created but "
23                  << "not yet have details." << endl;

```

```

24     }
25
26     Automobile(string a, int b, double c)
27     {
28         make = a;
29         model = b;
30         price = c;
31         cout << "Automobile object: " << make << " makes in "
32             << model << ". The price is RM " << price << "."
33             << endl;
34     }
35
36     void printInfo(){ }
37 }; //End class Automobile
38
39 class Car: private Automobile
40 {
41     private:
42         int doors;
43
44     public:
45         Car(): Automobile("BMW", 2010, 150000.0)
46         {
47             doors = 0;
48             cout << "Car with car's make, year model and price "
49                 << "(" << "BMW", 2010, 150000.0) accordingly."
50                 << endl;
51         }
52
53         Car(string a, int b, double c, int d): Automobile(a, b, c)
54         {
55             doors = d;
56             cout << a << "'s car has " << d << " doors." << endl;
57         }
58
59         void printCar() { }
60 }; //End class Car
61
62 class Truck: protected Car
63 {
64     protected:
65         string driveType;
66
67     public:
68         Truck(string d): Car("Toyota", 2014, 45000.0, 4)
69         {
70             driveType = d;
71             cout << "This is the truck with " << d
72                 << " drive type." << endl;
73         }
74
75         void printTruck() { }
76 }; //End class Truck

```

```

78
79 int main()
80 {
81     cout << fixed << setprecision(2);
82
83     Car car;
84     Truck truck("4WD");
85     Automobile automobile;
86
87     return 0;
88 }

```

- a. What will the lines 83 to 85 print onto the screen when the program runs?
- Car car;
 - Truck truck("4WD");
 - Automobile automobile;
- b. Based on the objects created in the main function of the, determine the member variables that each object owns. Write (**Own**) in the corresponding blank cells, if the object owns or has a copy of the variable; and write (**No**) if otherwise.

	Member Variables				
Object	make	model	price	doors	driveType
car					
truck					
automobile					

- c. With reference to the same member variables in (a), determine whether they are accessible inside the methods and by the object listed below. Write (**Accessible**) in the corresponding blank cells, if it is accessible; and write (**No**) if otherwise.

	Member Variables				
Object	make	model	price	doors	driveType
printInfo()					
printCar()					
printTruck()					
automobile					

5. Compile and run Program 7.5. Understand what each line of the statement in the program does. Based on the program, draw the class diagram. In your own words, explain why such output is printed.

```

1 //Program 7.5
2 #include <iostream>
3 using namespace std;
4 class Transport {

```

```

5     public:
6         Transport();
7         Transport(const Transport&);
8     };
9
10    Transport::Transport() {
11        cout<<"Transport default constructor is called"<<endl;
12    }
13
14    Transport::Transport(const Transport& trans) {
15        cout<<"Transport copy constructor is called"<<endl;
16    }
17
18    class Car: public Transport {
19    public:
20        Car() {
21            cout<<"Car default constructor is called"<<endl;
22        }
23
24        Car(const Car& car) {
25            cout<<"Car copy constructor is called"<<endl;
26        }
27    };
28
29    class Motor: public Transport {
30    public:
31        Motor() {
32            cout<<"Motor default constructor is called"<<endl;
33        }
34
35        Motor(const Motor& motor) {
36            cout<<"Motor copy constructor is called"<<endl;
37        }
38    };
39
40    int main() {
41        Car car1;
42        Motor motor1;
43        Transport trans = car1;
44        return 0;
45    }

```

6. Compile and run Program 7.6. Understand what each line of the statement in the program does. Based on the program, draw the class diagram. In your own words, explain why such output is printed.

```

1    //Program 7.6
2    #include <iostream>
3    using namespace std;
4    class Thought{
5    public:
6        void message() {
7            cout<<"Some people get lost in thought"<<endl;

```

```

8      }
9  };
10
11  class Advice: public Thought{
12      public:
13          void message(){
14              cout<<"Avoid cliches like the plague"<<endl;
15          }
16  };
17
18  int main(){
19      Thought think;
20      Advice cliche;
21      think.message();
22      cliche.message();
23      return 0;
24  }

```

Now, run the following Program 7.7. Why such output is printed?

```

1  //Program 7.7
2  #include <iostream>
3  using namespace std;
4  class Thought {
5      public:
6          void message() {
7              cout<<"Some people get lost in thought"<<endl;
8          }
9  };
10
11  class Advice:public Thought {
12      public:
13          void message() {
14              Thought::message();
15              cout<<"Avoid cliches like the plague"<<endl;
16          }
17  };
18
19  int main() {
20      Thought think;
21      Advice cliche;
22      think.message();
23      cliche.message();
24      return 0;
25  }

```

7. Based on Program 7.8:
 - a. Identify which showValue function applies the overriding and overloading concept.
 - b. Trace the output of the program.
 - c. Define the difference between method overriding and method overloading.


```

1 //Program 7.8
2 #include <iostream>
3 using namespace std;
4
5 class Superclass3{
6     public:
7         void showValue(int a){
8             cout<<a<<endl;
9         }
10 };
11
12 class Subclass3:public Superclass3{
13     public:
14         void showValue(int a){
15             cout<<"The subclass: \t"<<a<<endl;
16         }
17
18         void showValue(double a){
19             cout<<a<<endl;
20         }
21 };
22
23 int main(){
24     Subclass3 obj;
25     obj.showValue(8);
26     obj.showValue(8.97);
27     return 0;
28 }

```

8. Consider the following two classes in Program 7.9.

```

1 //Program 7.9
2 #include<iostream>
3 using namespace std;
4
5 class B{
6     private:
7         int k;
8
9     protected:
10         int i;
11         int j;
12         void printMe();
13
14     public:
15         B(int j){
16             this->j=j;
17         }
18
19         void changeValue(int);
20 };
21
22 void B::printMe(){

```

```

23     cout<<"i:"<<i<<" j:"<<j<<" k:"<<k<<endl;
24 }
25
26 void B::changeValue(int j){
27     this->j = j;
28 }
29
30 class C:public B{
31     private:
32         int j;
33
34     public:
35         C(int value){
36             i = 6;
37             j = 10;
38         }
39         void printMe();
40
41     protected:
42         void changeValue(int);
43 };
44
45 void C::printMe(){
46     B::printMe();
47     cout<<"j: "<<j;
48 }
49
50 void C::changeValue(int j){
51     B::changeValue(j);
52     this->i=25;
53 }
54
55 int main(){
56     C obj(12);
57     obj.changeValue(45);
58     obj.printMe();
59 }

```

The Program 7.9 will result in a compilation error. You are required to:

- Identify which line(s) of code result in s compilation error(s).
- Give a reason(s) for the error(s).
- Correct the statement(s).

Write the output of the program once the error(s) have been corrected.

9. Given Program 7.10 that uses multiple inheritance:

```

1 //Program 7.10
2 #include <string>
3 #include <iostream>
4 using namespace std;

```

```

5
6 class Person
7 {
8     private:
9         string name;
10        int age;
11        bool IsMale;
12
13    public:
14        Person(string name,int age,bool bIsMale:
15            name(name),age(age),IsMale(bIsMale){}
16
17        string getName() { return name; }
18        int getAge() { return age; }
19        bool getIsMale() { return IsMale; }
20 };
21
22 class Employee
23 {
24     private:
25         string m_strEmployer;
26         double m_dWage;
27
28     public:
29         Employee(string strEmployer, double dWage):
30             m_strEmployer(strEmployer), m_dWage(dWage){}
31
32         string GetEmployer() { return m_strEmployer; }
33         double GetWage() { return m_dWage; }
34 };
35
36 class Teacher: public Person, public Employee
37 {
38     private:
39         int m_nTeachesGrade;
40
41     public:
42         Teacher(std::string strName, int nAge, bool bIsMale,
43             string strEmployer, double dWage, int nTeachesGrade):
44             Person(strName, nAge, bIsMale), Employee(strEmployer,
45             dWage), m_nTeachesGrade(nTeachesGrade){}
46
47         void display(){
48             cout<<"Name:\t"<<Person::getName()<<endl;
49             cout<<"Age:\t"<<Person::getAge()<<endl;
50             cout<<"Employer:\t"<<Employee::GetEmployer()
51                 <<endl;
52             cout<<"Wage:\t"<<Employee::GetWage()<<endl;
53             cout<<"Grade:\t"<<m_nTeachesGrade<<endl;}
54 };
55
56 int main(){
57     Teacher teach1 ("Allan", 35, true, "UTM", 4500, 45);

```

58	teach1.display();
59	return 0;
60	}

- a. Save Program 7.10 as `PersonRelation.cpp`. Then compile and run Program 7.10. Understand what each line of the statement in the program does. Based on the program, draw the class diagram. In your own words, explain why such output is printed.
- b. Now, remove the `Person` and `Employee` classes from the Program 7.10 and place the classes in their own files. Use `#include` at the top of the `PersonRelation.cpp` file to include the two parent classes (eg.: `#include "person.cpp"`). Compile and run the program. You will get the same output as Program 7.10.

10. Create a class **B** that has the following:

- a. `m`, an integer. This member variable should not be accessible to code outside the class or to any its subclass.
- b. `n`, an integer. This member variable should be accessible only to its subclass.
- c. `setM`, `getM`, `setN` and `getN`. These are the mutator and accessor methods for the member variables `m` and `n`. These methods should be accessible to code outside the class.
- d. `calc`. A public method that returns the value of `m` multiplies `n`.
- e. Create a class **D** that is a subclass to **B**. The class members are:
 - i. `q`, a double. This member variable should not be accessible to code outside the class.
 - ii. `r`, a double. This member variable should be accessible only to its subclass.
 - iii. `setQ`, `setR`, `getQ` and `getR`. These are the mutator and accessor methods for the member variable `q` and `r`. These methods should be accessible to code outside the class.
 - iv. `calc`. A public method that overrides the superclass `calc` method and return the value of `q` multiplies `r`.
- f. Create a main method to test the functionality of your **B** and **D** classes.

11. Examine the following class declarations in Program 7.11:

1	//Program 7.11
2	class Faculty
3	{ protected:
4	string facName;
5	string buildingNo;
6	
7	public:
8	Faculty (string,string);
9	};

```

10
11 Faculty::Faculty(string facName, string buildingNo){
12     this->facName=facName;
13     this->buildingNo=buildingNo;
14 }
15
16 class Student
17 { protected:
18     string name;
19
20     public:
21     Student(string name)
22     { this->name=name;}
23 };

```

Write the declaration of a class named TeachingAssistant. The class should be derived from both the Faculty and Student classes above. This should be a case of multiple inheritance, where both Faculty and Student are base classes. Draw its complete class diagram.

- a. The TeachingAssistant class will have an attribute course of type string.
 - b. The TeachingAssistant class will also call constructors of its base classes.
12. Based on the following UML class diagram in Figure 7.1, write a complete C++ program that implements the relationships among classes. The explanation of each method is as in Table 7.2.

Table 7.2: Explanation of each method in each class

Methods	Explanation
Flight class	
addPassenger (Passenger)	This method will add Passenger objects to the passengerList.
printInfo()	This method will display all flight information as an example below: <pre> Flight No: MH3120 Destination: Johor Bahru Departure: 8:10 Arrival: 9:00 Number of Passengers: 10 </pre>
Passenger	
printDetails()	This method will display all passenger information, i.e. name and age.
Kids class	
printDetails()	- This method will display all kids passenger information. - This method overrides its superclass printDetails method. The values of its

	inherited attributes will be displayed by calling its inherited method.
Adult class	
<code>printDetails()</code>	- This method will display all adults passenger information. - This method overrides its superclass <code>printDetails</code> method. The values of its inherited attributes will be displayed by calling its inherited method.
Time class	
<code>getHour()</code>	This method will return the attribute value of hour.
<code>getMinute()</code>	This method will return the attribute value of minute.

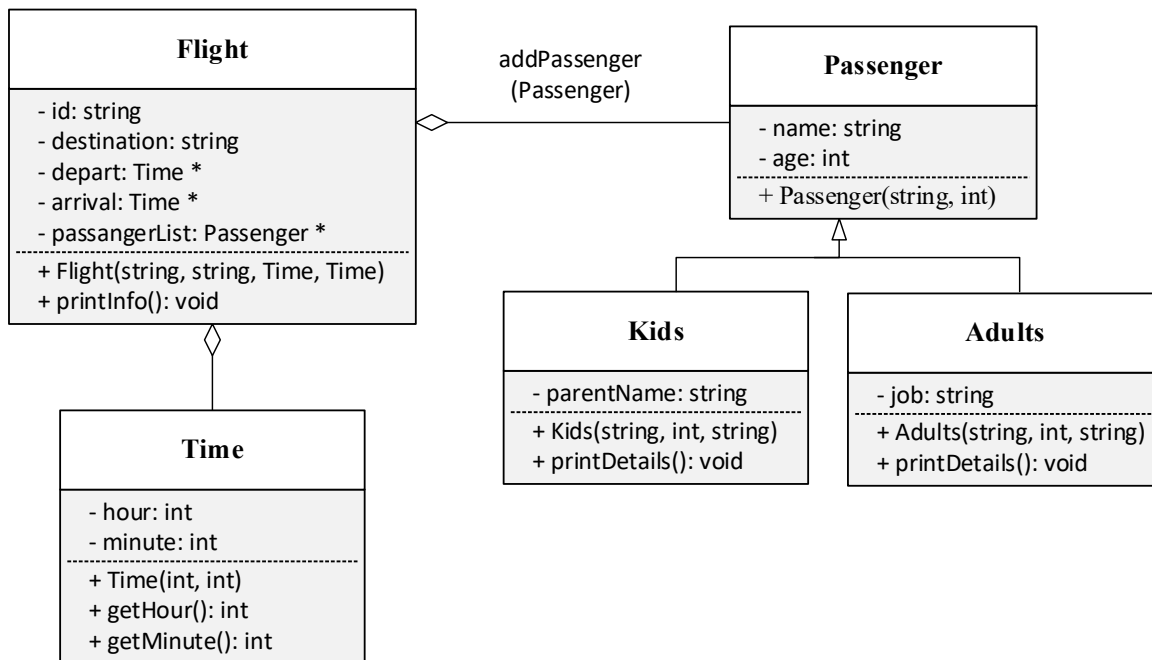


Figure 7.1: Class Diagram of Flight, Passenger, Kids, Adult and Time classes

Your program must be able to produce the following output as illustrated in Figure 7.2.

```

Flight Number:  NAS7921
Destination:    Johor Bahru
Departure:      8:10
Arrival:        9:00

Number of Passengers: 3
Number of Adults:    2
  
```

Number of Kids:		1			
Passengers Details:					
Name:	Ali	Age:	34	Job Title	Lecturer
Name:	Goh	Age:	25	Job Title	Student
Name:	Azmi	Age:	3	Parents Name:	Ali

Figure 7.2: Example of expected output

13. Faculty of Computing will be organizing a competition that is open to all UTM students. The competition is meant to provide a platform to the students to exhibit their idea. Some important notes regarding the competition:
 - a. It is an individual participation.
 - b. Each student must have a project beforehand.
 - c. Each student must find an advisor which can be any UTM staff regardless of their position. That means, besides lecturers, technicians, research officers, etc, also can be appointed as advisers. However, only one advisor is allowed for each student.
 - d. Also, each staff can only advise a student.

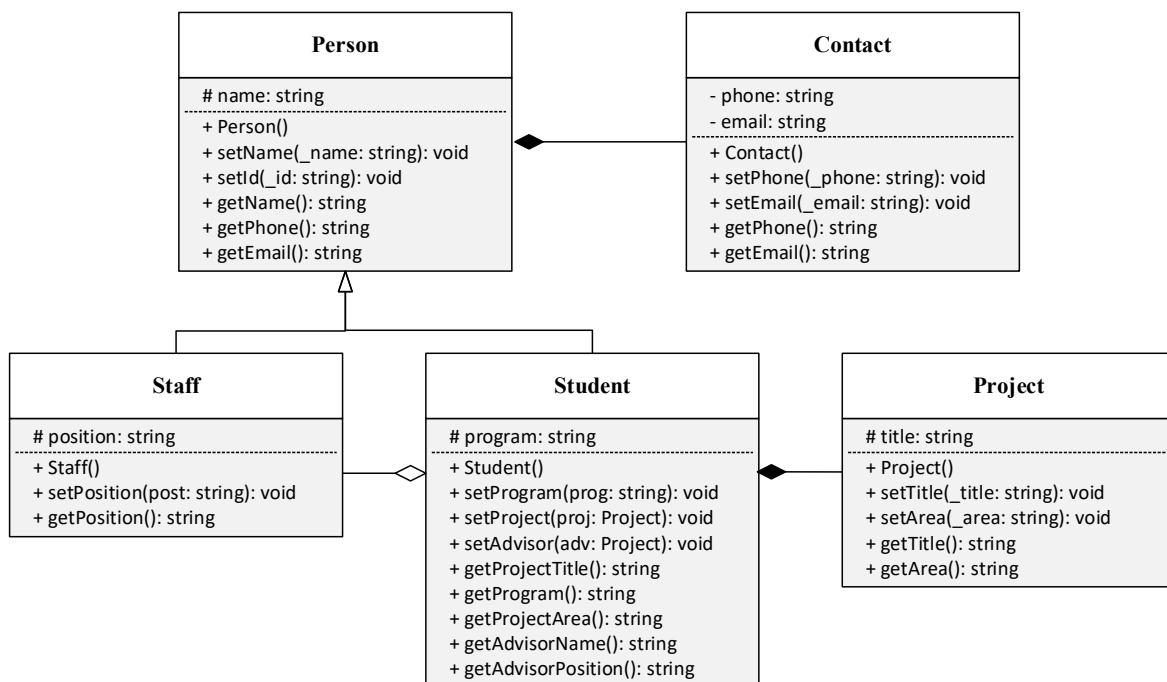


Figure 7.3: UML class diagram

Suppose the class design has been done by other parties as shown in Figure 7.3. Based on the class diagram in Figure 7.3 and the requirements above, write a C++ program that does the following tasks:

- a. Implement all the classes and their relationships. You need to define all the methods listed in the diagram, and you can add other methods whenever possible.
- b. Create a list of `Advisor` objects and another list of `Student` objects. Use the sample data given in Table 7.3 and Table 7.4.
- c. Assign each student with an advisor.

- d. Print the information of all the students along with their projects and advisers.
- e. Suppose the first and second students want to change their advisor each other, while the advisor for the third student wants to withdraw from being appointed as an advisor. Write the code to implement these situations.
- f. Print again the final list.

Table 7.3: Sample data of advisors

No	Advisor's Name	Phone	Email	Position
1	Prof. Dr. Muhammad Roslan Abdullah	0199875678	mroslana29@gmail.com	Lecturer
2	Mr. Qamarool Zaman	01156781234	qam23@gmail.com	Senior Technician
3	Dr. Siti Zubaidah Rosli	0101117456	ctzr983@gmail.com	Research Officer

Table 7.4: Sample data of students

No	Student's Name	Phone	Email	Program	Project
1	Lim Sew Ying	010897634045	lsying12@live.utm.my	Electrical Engineering	Smart Dustbin
2	Abu Bakar Razali	0111210000	abraz78@live.utm.my	Computer Science	Money Recognition for Blind People
3	Nur Amalina Muhammad	0129000123	nam978@live.utm.my	Biomedical Engineering	Dengue Testing Kit